

BIT MESRA Invigilation Scheduler

Future Decoupled Architecture, Database Design & Cloud Deployment Blueprint

1. Production Decoupled Architecture

To support multiple concurrent administrators, prevent request timeouts during complex solver executions, and scale resources, we must migrate to a decoupled cloud-ready architecture:

- **Frontend Client (Vite + React.js):** Modern SPA client hosted on Amazon S3 and distributed via CloudFront CDN. Handles interactive grids, live diagnostics visualization, and JSON configurations state locally.
- **Backend API Server (FastAPI):** High-performance asynchronous API server processing user authorizations, database operations, and job dispatching. Generates automatic Swagger/OpenAPI documentation.
- **Task Queue (Celery + Redis):** Solver engine tasks are decoupled. FastAPI dispatches solver jobs to Redis (broker). A pool of Celery workers picks up the tasks asynchronously, executes optimization search, and saves results back to the database.

2. Database Engine & Schema Design

Database Selection: PostgreSQL (via Amazon RDS) is chosen for its robustness, native JSONB support (useful for dynamic settings configurations), transactional safety (ACID compliance), and compatibility with SQLAlchemy ORM.

- **Model mapping:** Transition configurations and states to SQL tables:
 - faculties (stores emp codes, names, standardized categories, PG timetable exclusions).
 - sessions (stores slots details, duration hours, day indices, workloads).
 - schedules (stores calculated invigilator allocation details, coverage percents).
 - history (persists historical imbalances to allow continuous load balancing over multiple semesters).

3. Step-by-Step Deployment Pipeline

The deployment steps are automated using Continuous Integration/Continuous Deployment (CI/CD):

Step A: Containerization

Write multi-stage Dockerfiles to build optimized, lightweight images for the Frontend (distributing static assets via Nginx), API Server (FastAPI with production uvicorn server), and Worker Nodes (Celery task executor).

Step B: Infrastructure provisioning (IaC)

Use Terraform to provision AWS resources inside a custom VPC: public subnets for the Application Load Balancer (ALB), and private subnets for ECS Fargate containers, Redis Cache, and RDS PostgreSQL database.

Step C: CI/CD Pipeline (GitHub Actions)

On code push: (1) Run unit tests & code linters, (2) Build Docker images, (3) Push images to Amazon Elastic Container Registry (ECR), (4) Trigger ECS Fargate rolling updates to deploy code without downtime.

4. Implementation Roadmap Timeline

Phase	Target Focus	Key Deliverables
Phase 1	Backend Refactoring	FastAPI REST endpoints, Celery workers + Redis setup
Phase 2	Database & ORM	RDS PostgreSQL schema migrations via Alembic
Phase 3	SSO & Admin UI	JWT auth, LDAP/Active Directory SSO integration
Phase 4	DevOps & Launch	Terraform provision, GitHub Actions CI/CD setup